



Practice Test 2: Answers and Explanations

PRACTICE TEST 2 ANSWER KEY

- | | |
|-------|-------|
| 1. D | 22. D |
| 2. D | 23. D |
| 3. D | 24. C |
| 4. C | 25. B |
| 5. B | 26. A |
| 6. D | 27. A |
| 7. C | 28. D |
| 8. A | 29. D |
| 9. D | 30. D |
| 10. A | 31. D |
| 11. C | 32. D |
| 12. B | 33. B |
| 13. B | 34. C |
| 14. C | 35. B |
| 15. B | 36. C |
| 16. A | 37. A |
| 17. D | 38. C |
| 18. A | 39. C |
| 19. B | 40. C |
| 20. D | 41. C |
| 21. D | 42. A |

PRACTICE TEST 2 EXPLANATIONS

Section I: Multiple-Choice Questions

1. **D** The question involves integer division, modulus division, and order of operations. One important thing to remember is integer division is performed if both sides of the division operator (/) are integers, i.e., no rounding will occur. Another is that division, multiplication, and modulus division (%) are all performed left to right.

Statement (I) will perform integer division first: $24 / 5 = 4$. Next, $4 \% 3 = 1$. Finally, $1 * 2 = 2$. This solution is less than 5 so (I) must be part of the answer. Eliminate (B).

Statement (II) will also perform integer division first: $12 / 3 = 4$. Next, $4 * 2 = 8$. Finally, $8 + 1 = 9$. This solution is not less than 5, so (II) must not be part of the answer. Eliminate (C).

Statement (III) will perform mod first: $4 \% 3 = 1$. Next, $1 * 2 = 2$. Finally, $1 + 2 = 3$. This solution is also less than 5, so (III) must be part of the solution. Eliminate (A).

Since (I) and (III) yield solutions less than 5, the correct choice is (D).

2. **D** The code `"\n"` is an escape character or line break in Java, causing the printing to begin a new line. This eliminates (A) and (B) since those choices print `\n` instead of skipping to a new line. Additionally, the backslash character (`\`) is used to indicate when a special character such as a `"` or `\` is to be printed: `\"` will print a double-quote (`"`) and `\\` will print the backslash. The forward slash (`/`) does not need any special character to print, thus the `//` will print two forward slashes as seen in (D). Choice (C) is not correct as it only has one forward slash. Choice (D) is correct.
3. **D** This question involves matching data types of the arguments with parameters expected by the constructors. Choice (A) uses the first constructor on lines 7–12. An integer such as 60 may be used in a field defined as double. Choice (B) again uses the same constructor, this time by casting 13.0 to an int field. Choice (C) uses the constructor on lines 13–17. One might think the `toString()` method may cause a `NullPointerException` because the name was not initialized. That exception would not occur when printing the field, only if trying to use the field, such as in a decision statement looking for a value other than `null`. Choice (D) will cause an error because there is no constructor expecting a double data type as the third parameter. The answer is (D).

4. **C** The client method header for `computeBonus` has two parameters:

```
computeBonus(SalesRep s, double percentage)
```

The instance field `ytdSales` within the `SalesRep` object is private, so the data must be referenced using an accessor: `objectName.methodname()`. Thus, to obtain `ytdSales` for the object named `s`, we must use `s.getYtdSales()`, eliminating (A) and (B). The field `percentage` is a parameter provided to the method, not a public instance field of the object, so there is no need to use `s.percentage` as represented in (D). Thus (C) is the correct answer.

5. **B** Choice (A) is incorrect because it uses `ArrayList` as the data type in the enhanced-for loop, rather than `SalesRep` (the type of objects in the `ArrayList`). Choice (C) is incorrect because it uses the data type with `.get(index)` (it should use the name of the `ArrayList` with `.get(index)`). Choice (D) accesses the `ArrayList` and method correctly; however, the loop goes out of bounds because the last index should be one less than the size of the `ArrayList`. Choice (B) uses an enhanced-for loop (which will not go out of bounds) and accesses the `ArrayList` and method correctly. The answer is (B).
6. **D** When using the `substring` method, the first character of a string is at index 0. The parameters of the `substring` command are the first index and ending index, but the command stops just before the ending index is reached, so `substring(1, 3)` will print characters at indexes 1 and 2 ("up"). This eliminates (A) and (C). The command `str.substring(1)` accesses the first character to the end of the string ("uperstar"), but it is NOT assigned back into `str`. Since strings are immutable, `str` is not modified (on the third or fifth lines)—"up" will be printed three times. The answer is (D).
7. **C** Statement (I) will successfully alternate the value of `isBlack`. The right side of the assignment statement will change the value to the opposite value every time the code is executed. Eliminate (B). Statement (II) specifically checks whether the value of `isBlack` is false. If so, it will change the value to true. The else will only be executed if `isBlack` is true, in which case it will change the value to false, so (II) is successful at alternating values, as well. Eliminate (A) and (D). Only one choice remains, so there is no need to continue. However, note that (III) only changes the value of `isBlack` if it is true: there is no provision to change the value from false to true, so Statement (III) is incorrect. Since only (I) and (II) would successfully alternate the values, the answer is (C).
8. **A** Choice (A) would simply compare the object references of the two `String` variables, not the actual contents of the fields. Both (B) and (C) are perfectly straightforward methods to compare the contents of `String` variables. Choice (D) utilizes the `substring` method two different ways to examine the entire string for both `str1` and `str2` with the `.equals()` method being used to compare the contents of the strings. Thus, (A) is the correct answer.
9. **D** Start with De Morgan's Law.
- The original statement is $!(\neg p \parallel q) \parallel \neg(p \parallel \neg q)$
- After De Morgan's Law $(p \ \&\& \ \neg q) \parallel (\neg p \ \&\& \ q)$
- Examining the conditions on either side of the `||` operator, the truth values of `p` and `q` are combined with an `&&` operator and are direct opposites. Thus, if the two values are direct opposites of each other on either side of the `||` operator, the entire expression would be true. The correct choice is (D).
10. **A** Short-circuit evaluation is used when the first part of an `&&` condition is false (since both sides of an `&&` condition must be true, a false in the first part causes the whole expression to be false). Both (C) and (D) start with an `&&` operator. Since neither has a false value in `p`, short-circuit cannot be used on these statements.

Alternatively, short-circuit can be used if the first part of an `||` condition is `true` (since only one side of an `||` condition needs to be `true`, a `true` in the first part causes the whole expression to be `true`). Since `p` has a `false` value in (B), a short-circuit cannot be used on this statement.

Choice (A) has a `true` value as the first part of an expression with the `||` operator, making the whole statement evaluate to `true`. The answer is (A).

11. **C** Choices (A), (B) and (D) will all generate compile time errors. The correct formats are:

```
File <file name>=new File(<file path>);
Scanner <scanner name>=new Scanner(<file name>);
```

The correct answer is (C).

12. **B** The outer loop will execute 10 times, where `i` will range from 1 to 10 (`< 11`) incrementing by one each time. The inner loop will execute 5 times, where `j` will range from 10 to 2, decreasing by 2 each time and stopping after `j` is no longer greater than 1. The variable `count` will be incremented by 1 each time the inner loop executes, so $(10 * 5) = 50$ times. The variable `star` starts with one `"**"` but adds two more (`"**"`) each time the inner loop executes. $(1 + 50 * 2) = 101$. The final length of `star` will be 101. Thus, (B) is the correct answer.

13. **B** The outer loop will execute 5 times (`i` is initialized to 1, the loop will continue until `i` is no longer `<= 5`). Within the outer loop are two loops, each using `print` statements so the output will continue across a single line. After the two loops execute, a `println` statement will advance the print to the next line.

The first inner loop initializes `j` to 1 and continues while it is less than `i`. So, on the first row, this loop doesn't execute at all because `j` and `i` are both 1 and `j` is not less than `i`. The `"- "` is never written, thus eliminating choice (A).

The second loop initializes `j` as the same value as `i`, continuing, up to 5, thus generating 5 `"* "`, eliminating choice (C).

Each time the outer loop executes, one additional `"- "` is written in the first loop, and one fewer `"* "` is written in the second loop. Choice (D) writes the correct number of each symbol, but in reverse order. The `"- "` is written in the first inner loop. Thus, choice (B) is correct.

14. **C** Three trace tables will make short work of this problem.
In (I), the variable `i` will have the values 1 and 2; the loop will stop when `i` increments to 3.

<code>i</code>	<code>2 * i + 1;</code>	<code>sum</code>
1	<code>2 * 1 + 1 = 3</code>	3
2	<code>2 * 2 + 1 = 5</code>	8

In (II), the variable i will have the values 1–5, the loop will stop when i increments to 6.

i	if ($i \% 2 == 1$)	sum
1	$1 \% 2 == 1$, so add i to sum	1
2	$2 \% 2 == 0$	1
3	$3 \% 2 == 1$, so add i to sum	4
4	$4 \% 2 == 0$	4
5	$5 \% 2 == 1$, so add i to sum	9

In (III), the variable i will start at 5; then the loop will change i to 3, then 1. The loop will not be re-entered when i is 1.

i	Initial values, before the loop begins	sum
5		5
3	sum += i	8
1		9

Using the trace tables above, (C) is the answer because (II) and (III) both print 9, while (I) prints 8.

15. **B** Segment (I) starts the loop with k equal to 1, increments k by 1, then adds k to sum. The variable k is added whether it is even or odd, thus this option is unacceptable. Eliminate (A) and (D).

Segment (II) starts the loop with k equal to 1, increments k by 2, ensuring only odd numbers will be added. The loop continues while k is less than or equal to `number`. If `number` is odd, the last value of k be equal to `number`, while if k is even, the last value of k to be added to the sum will be the previous odd number. Only odd integers up to `number` will be added. This segment works properly. Eliminate (C).

Three choices have been eliminated, so there is no need to go further. However, to see why (III) does not work, note that it starts the loop with k equal to `number` and decrements k by 2. If `number` is odd, this option will work as k will always be odd. However, if `number` is even, even numbers will be added, which does not meet the criteria of the method.

Since only (II) works properly, the correct choice is (B).

16. **A** Options I and II only.

Option I is correct. The code splits the line into the word and its synonyms using the delimiter ":", creating the array `temp`. The array `temp` contains the word in `temp[0]` and the synonyms in `temp[1]`. `temp[1]` is passed to the method `separateSynonyms` which

- creates the `ArrayList<String> syns`
- splits the list of synonyms using the delimiter ":", creating the array `temp`
- adds the elements of `temp` to `syns`
- returns `syns`

The `Synonym` object `syn` is constructed from `temp[0]` and the `ArrayList` created above.

Option II is correct. The code separates the line into two substrings: `str1` is the substring before the ":" (the word), and `str2` (the synonyms) is the substring after the ":". `str2` is passed to the method `separateSynonyms` which is described above. The `Synonym` object `syn` is constructed from `str1` and the `ArrayList` created above.

Option III is incorrect. It passes the entire line to the method `separateSynonyms` which means that the word is included along with the synonyms.

The correct answer is (A).

17. **D** Choice (A) works, as the static fields of the `Integer` class, `Integer.MIN_VALUE` and `Integer.MAX_VALUE`, represent the range of numbers that can be represented in 4 bytes of data (the size of an `int` field). Since (A) does not cause an error, it can be eliminated. The smallest number an `int` field can hold is the value held in `Integer.MIN_VALUE`: thus, when 1 is subtracted, an overflow error occurs, resulting in `Integer.MAX_VALUE`. The result is unexpected, but no error is thrown by the compiler or at run time. Choice (B) can be eliminated. In (C), the fields are added, producing -1, not causing any error at all. Choice (D) will cause a compiler error because the value being assigned is 1 larger than the limit of what an `int` data field can store.
18. **A** The method `half` divides the parameter and returns the result of dividing the parameter by 2. Since neither `n` nor 2 is a double data type, only integer division occurs. Thus, when `n` is 5, `n / 2` will be 2, but 2.0 is returned because the return type was specified as double. The contents of `num` remain the same (5). The `print` statement starts with the empty string (""), which will cause the results of `half(num)` and `num` to be concatenated together as strings. Thus, the output is 2.05 (where 2.0 and 5 have been concatenated together as a string). Choice (A) is correct.
19. **B** The `getPerimeter()` method returns a double, so (A) and (C) can be eliminated. The `MyRectangle` constructor creates a local variable `perimeter`, which has the same name as the instance variable. When this occurs and the keyword `this` has not been used, the local variable is accessed. Thus, the local variable `perimeter` is assigned the value of 10, but the instance field only contains a default value of 0. When the `getPerimeter()` method returns the value of `perimeter` as a double, it returns 0.0 because the instance field `perimeter` has never been assigned any other value. Choice (D) is eliminated. The answer is (B).
20. **D** Choice (D) uses the same number and the same type of parameters in the same order as the existing constructor: `String, int, boolean, int`. This is not allowed in Java because it would be impossible to know which constructor to invoke. All the other choices would be acceptable because they have different numbers of parameters. The answer is (D).

21. **D** The `Car()` constructor has no parameters, so all primitive fields will be initialized to 0 or false appropriately, but the `String` field will be initialized to `null`. The `setModel` method does not initialize the instance field `model`. When the parameter has the same name as the instance field, the keyword `this` must be used, otherwise the parameter is simply being assigned to itself. When the `if` statement attempts to compare the contents of `model` to "Tacoma", the `model` contains `null` so a `NullPointerException` will be thrown. Choice (D) is correct.
22. **D** The field `mpg` is supposed to be rounded to the nearest integer. Choices (A) and (B) produce real numbers without casting to `int` so they can be eliminated. The division in (C) produces a double but casts the double to an integer field too early. When 0.5 is added, the calculation has once again produced a double. Choice (D) performs regular division, then correctly adds 0.5 before casting to an integer. Casting to integer will truncate the decimal portion of the number. This algorithm will round the answer correctly. The answer is (D).
23. **D** Start by determining the possible range of values that may be generated in each variable. `Math.random()` generates values from 0 to just under 1 (use .99 for simplicity).
- ```
int a = (int) (Math.random() * 4) + 1; // may generate values from 1 to 4
int b = (int) (Math.random() * 5) + 3; // may generate values from 3 to 7
```
- Determine the range of values that may be produced by `a - b`.
- First use the smallest then largest values of `a` and subtract the lowest value of `b`.  $(1 - 3)$  through  $(4 - 3)$ . This range produces  $-2$  through  $1$ :  $-2, -1, 0, 1$ .
- Eliminate (B) and (C).
- Next, use the smallest then largest values of `a` and subtract the highest value of `b`.  $(1 - 7)$  through  $(4 - 7)$ . This range produces  $-6$  through  $-3$ :  $-6, -5, -4, -3$ .
- Eliminate (A).
- The correct choice is (D) because 2 cannot be generated by the problem.
24. **C** The parameters of the `substring` are the beginning index and ending index. The expression `str1.substring(1,4)` will return the characters starting at index 1, going up to but not including 4: "tar". When `substring` is used with only one parameter, the returning string will start at the position of that parameter and continue to the end of the string. The expression `str2.substring(5)` will return "ray". Concatenated together, the string will be "tarray". The correct choice is (C).
25. **B** The variable name has private access to the `Entertainer` class. An accessor would have to be implemented in the `Entertainer` class. Therefore, the answer is (B).



26. **A** An array is passed by reference to a method, so any changes that are enacted upon the array in the method are actually changing the array itself. The array is initialized with 5 elements and each element is assigned the index multiplied by 5, so (A) is correct. Choice (B) is incorrect because it assumes the array was unchanged by the method and allows for 6 elements rather than 5. Choice (C) is incorrect because it again allows for 6 elements, although they would reflect the index multiplied by 5. Choice (D) is incorrect because the array is defined using the primitive `int`, so it was initialized with 0s until it later is assigned the multiples of 5. The answer is (A).
27. **A** It is helpful to label the rows and columns with their indexes to assist with the math operations.

|      |   | Columns |   |   |   |
|------|---|---------|---|---|---|
|      |   | 0       | 1 | 2 | 3 |
| ROWS | 0 | 1       | 2 | 3 | 4 |
|      | 1 | 1       | 3 | 5 | 7 |
|      | 2 | 2       | 4 | 6 | 8 |
|      | 3 | 4       | 3 | 2 | 1 |

A single loop controlled by `j` runs from 0 to less than the number of rows (4)  
The calculations for building the one-dimensional array are as follows:

```

result[0] = matrix[3][0] - matrix[0][3] 4 - 4 = 0
result[1] = matrix[3][1] - matrix[1][3] 3 - 7 = -4
result[2] = matrix[3][2] - matrix[2][3] 2 - 8 = -6
result[3] = matrix[3][3] - matrix[3][3] 1 - 1 = 0

```

Choice (A) is correct.

28. **D** `Math.random() * 100` generates real numbers between 0 and 100. It includes 0, and goes up to, but does not include, 100. `Math.random() * 100 + 1` will generate real numbers from 1, up to, but not including, 101. Choice (A) generates numbers of type `double`, which cannot be added to an `Integer ArrayList`. Choice (B) will generate numbers of type `int` ranging from 0 to 99 (`int` will truncate the decimal portion, not round it). Choice (C) will generate numbers between 0 and 100. Choice (D), which is the correct answer, will generate numbers between 1 and 100. The numbers are cast to (`int`), which can then be added to the `Integer ArrayList`. The answer is (D).
29. **D** The `ArrayList` is initialized to A B C D E. The first loop removes the entry at the following indexes, modifying the `ArrayList` as shown in the table below. It's important to remember that the `ArrayList` is shrinking in size: the loop will only carry out twice.

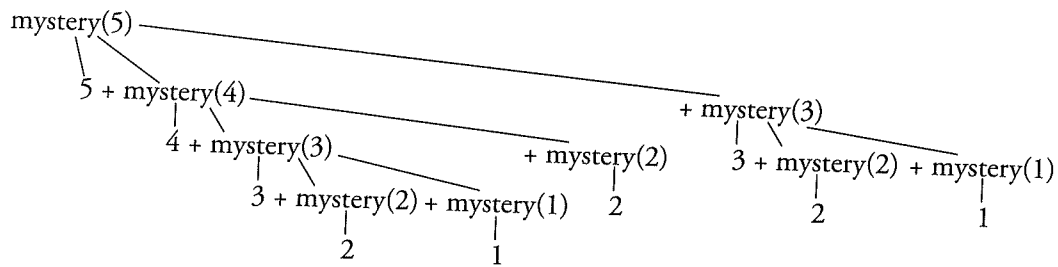
| Code                   | list1   |
|------------------------|---------|
| <code>remove(0)</code> | B C D E |
| <code>remove(2)</code> | B C E   |

The second loop inserts an "\*" three times, each time at index 1.

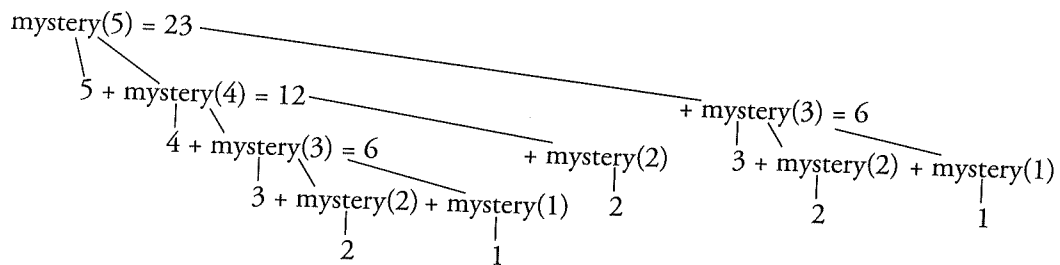
| Code                     | list1       |
|--------------------------|-------------|
| <code>add(1, "*")</code> | B * C E     |
| <code>add(1, "*")</code> | B * * C E   |
| <code>add(1, "*")</code> | B * * * C E |

The answer is (D).

30. **D** First, diagram the calls to the method, stopping each branch at its base case.



After diagramming the calls, add upwards.



31. **D** After the call `mystery("PLANT")`, `i` is 1,

```
temp = s.substring(s.length() - i);
s.substring(5 - 1);
s.substring(4); simply gives "T"
```

The length of `temp` on the next call is only 1 so nothing more is printed, and `mystery` is no longer called. Choice (D) is the correct answer.

32. **D** The `ArrayList` schedule contains objects that are `Play` data types. When `schedule[1]` is going to be printed, at run-time, Java looks to see whether the `object.toString()` method has been overridden. In other words, it looks to see whether there is a `toString()` method in the `Play` class. This method exists and does the following:

```
return performance + " with " + starringActor + " as " + mainCharacter;
```

The word "with" will be printed and does not exist in (A) and (C), so these choices can be eliminated. The first part of the return statement, returns `performance` is an object of type `Performance`. Again, Java looks to see whether there is a `toString()` method in the `Performance` class. This method also exists and does the following:

```
return name + " will be performed in " + season + " of " + year;
```

Since there is a `toString()` method defined, Java will use it instead of printing the memory location of the object, eliminating (B). The following will be printed:

Peter Pan will be performed in Spring of 2023 with Charlie as Peter

Therefore, the correct answer is (D).

33. **B** Choice (A) is incorrect because it tries to access the field `starringActor`, which is not visible because it is specified as a private variable in `Performance`. An accessor method is needed to use it. Choice (B) correctly uses an accessor method to return `starringActor` so that it may be added to the `actors ArrayList`. Choice (C) is incorrect because it uses the wrong data type in the enhanced-for loop: it needs to use `Play` instead of `Performance`. Choice (D) also tries to access `starringActor` directly but cannot because the field is private. Choice (B) is the correct answer.
34. **C** Option I incorrectly uses array notation `[i]` instead of the `get()` method as required for an array list. Eliminate (B) and (D). Both remaining choices include III, so you only need to look at II. Option II is correct; it correctly implements the `get()` method of an array list. Eliminate (A). Only one choice remains, so there is no need to continue. However, note that III incorrectly uses an enhanced-for loop to print the entire array list. The answer is (C).
35. **B** The number of searches needed to find a target value within a sorted array is  $2^n$ , where the value is just over the number of elements in the array. In this case,  $2^n = 4000$ .  $2^{11} = 2048$ , so (A) is not large enough. Choice (B) leads us to  $2^{12} = 4096$ , which will provide enough searches to find a target in 4000 elements. Choices (C) and (D) are too large. Choice (B) is the correct answer.
36. **C** The index used to control the outer loop runs from 1 to just under the length of the array. If an array had four elements, the indexes would be 0, 1, 2, 3. The loop would run from 1 to 3, which would be one less than the length of the array. The correct answer is (C),  $n - 1$ .
37. **A** The inner loop is controlled by:
- ```
while (j >= 0 && arr[j] > num)
```
- It is helpful to look at conditions in which the loop will stop executing or not execute at all. With the `&&` condition, both expressions have to be true, so look closer at the second part to determine when it might not be true. The logic surrounding the loop starts by assigning the second value in the array to `num`, then setting `i` to the previous location, resulting in two consecutive numbers in the array being compared. If those numbers happen to be the same, OR in ascending order, the loop will not execute at all. Thus, the correct answer is (A), 0 times.
38. **C** The expression `mat.length` returns the number of rows in a 2D array, while `mat[0].length` returns the number of columns in the first row. This code is selecting each row and moving columns within that row. The value in the first column is assigned to `temp`, while the loop travels to the end of the row shifting all elements to the left:

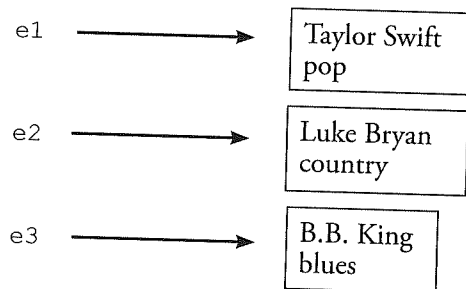
```
mat[a][b - 1] = mat[a][b];
```

When the row is completed, the value in `temp` is assigned to the last column:

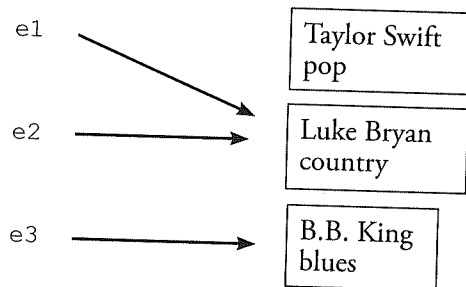
```
mat[a][mat[0].length - 1] = temp;
```

Choice (C) is the correct answer.

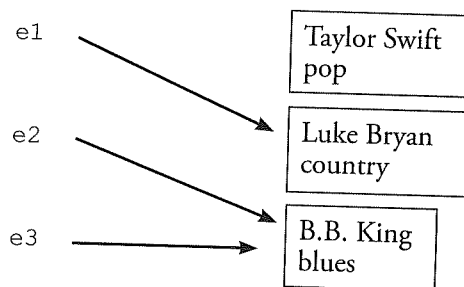
39. C After the declaration and initialization of `e1`, `e2`, and `e3`, the objects look like this in memory.



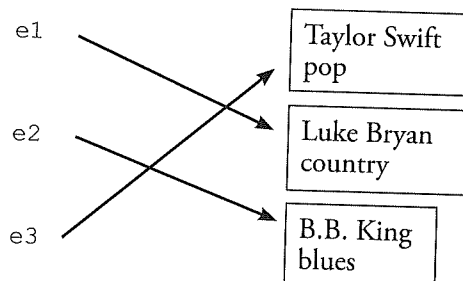
After `e1 = e2` executes, `e1` will have the same reference as `e2`.



After `e2 = e3` executes, `e2` will have the same reference as `e3`.



After `e3 = e1` executes, `e3` will have the same reference as `e1`.



The print statement will print Luke Bryan country, B. B. King blues, Luke Bryan country. Choice (C) is correct.

40. **C** Option (I) will remove some but not all the elements containing "Summer" due to the changes indexing as elements are removed within the loop.

b1: Spring 2022	b2: Summer 2022	b3: Summer 2022	b4: Summer 2022	s1: Winter 2024
index: 0	index: 1	index: 2	index: 3	index: 4

The first pass through the loop, the object at index 0 is examined. The season is not "Summer": the element is not removed.

The second pass through the loop, the object at index 1 is examined. The season is "Summer": the element is removed, and the indexes are updated.

b1: Spring 2022	b3: Summer 2022	b4: Summer 2022	s1: Winter 2024
index: 0	index: 2 1	index: 3 2	index: 4 3

The third pass through the loop, the object at index 2 is examined. But notice, b4 is being examined (b3 has been skipped because the indexes were updated after the remove). The object b3 will not be removed even though the season is "Summer" because it has been skipped. Therefore, (I) will not work. Eliminate (A) and (D).

An enhanced-for loop creates a local variable that does not reference an element in an array. It is assigned a value from the `ArrayList` and is used for read-only purposes. Thus, no elements in the `ArrayList` can be used with this method and Option (II) will not work. Eliminate (B).

Only one choice remains, so there is no need to continue. However, to see why (III) does work, note that it is similar to (I) but traverses the `ArrayList` backwards. Thus, when an element is removed from the `ArrayList`, the indexes update on elements that have already been processed. No elements containing "Summer" will be affected. Option (III) is the only option that will work properly, making (C) the correct choice.

41. **C** The + operator is used for both addition and concatenating strings. The expression within the parentheses is evaluated from left to right. If number data types are used, the + will be used as an addition symbol. As soon as a string is encountered, concatenation will be used. If number data types follow the string, Java will continue to concatenate instead of using addition. A way to think about it is, once concatenation starts, it will continue.

Evaluation:

```
String n = "1";
int sum1 = 0 + 1; // sum1 will store the integer 1
int sum2 = 1 + 1; // sum2 will store the integer 2
System.out.println(sum1 + sum1 + n + sum2 + sum2); // addition is performed
                2   + n + sum2 + sum2           // concatenation is performed
                2     1     2     2
```

The result is a string containing 2122. Choice (C) is correct.

42. **A** Start by evaluating value1. The method equals() evaluates the contents of phrase1 and phrase2 and they are exactly the same, so value1 will be true.

Next evaluate value2. The fields phrase1 and phrase2 are string literals. When Java creates these in memory, it will recognize that the string literal of both fields is exactly the same. Therefore, it will only create one string literal and will have both fields reference the same string literal. The == comparison operator will evaluate that both locations are the same and therefore true.

The correct choice is (A).

Section II: Free-Response Questions

```

1.  a.  public String convertWord(String word)
      {
        String w = word;
        String firstLet = w.substring(0,1);
        boolean firstIsVowel = isLetterAVowel(firstLet);
        if (firstIsVowel)
            return word + "way";
        String secondLet = w.substring(1,2);
        boolean secondIsVowel = isLetterAVowel(secondLet);
        if (secondIsVowel)
            return word.substring(1) + firstLet + "ay";
        return word.substring(2) + firstLet + secondLet + "ay";
      }

b.  public String convertPhrase()
      {
        if (phrase == null || phrase.length() == 0)
            return "";
        String p = phrase;
        String newPigLatinPhrase = "";
        String word = "";
        int j = 0;
        boolean isLastWord = false;
        int k = p.indexOf(" ");
        while (k != -1)
        {
            word = convertWord(p.substring(0, k));
            newPigLatinPhrase += word + " ";
            p = p.substring(k + 1);
            k = p.indexOf(" ");
        }
        if (p.length() > 0)
        {
            word = convertWord(p);
        }
        newPigLatinPhrase += word + " ";
        return newPigLatinPhrase;
      }

```

PigLatin Rubric

Part (a)

+4

+1

Correctly finds first character and second character.

+1

Checks whether the length of the word is 1 and whether it starts with a vowel; returns vowel + "way", otherwise returns word + "ay"

+1

Correctly converts word to PigLatin following the three rules

+1

Returns word converted to pigLatin, does not change the parameter: word

Part (b)

+3

+1

Checks for null or empty string. MUST check for null first. Returns empty string.

+1

Correctly finds a word and calls `convertWord(word)` for each word in the phrase, including the last word

+1

Concatenates all converted words that have been returned into a new pig latin phrase

2. Canonical solution:

```

public class FrequentFlyerMember {
    private int acctNumber;
    private String flyerName;
    private int lifetimeMiles;

    FrequentFlyerMember(int n, String name, int miles)
    {
        acctNumber = n;
        flyerName = name;
        lifetimeMiles = miles;
    }

    /** Adds miles to lifetimeMiles */
    public void addMiles(int miles)
    {
        lifetimeMiles += miles;
    }

    /** prints selected status information */
    public String getStatusInfo()
    {
        return acctNumber + " " + flyerName + " is a frequent flyer member with " +
            lifetimeMiles + " miles";
    }
}

```

FrequentFlyerMember Rubric

+7

- +1 class header must be: public class FrequentFlyerMember
- +1 four instance fields have been defined as private for flyer's number, name, lifetime miles, and status level as seen in canonical solution.
- +1 constructor header must have the same name as the class and 4 parameters(int number, String name, int miles)
- +1 the three instance fields have been correctly assigned
- +1 addMiles method header must correctly use return type void and an int parameter.
- +1 the parameter miles should correctly be added to the instance field lifetimeMiles
- +1 the getStatusInfo method must use String as a return type and have no parameters

3. FactorPair—Canonical solution

```

a. public ArrayList<FactorPair> buildArrayList(int n)
    {
        ArrayList<FactorPair> tempPairs = new ArrayList<FactorPair>();
        for (int i = 1; i < (n / 2); i++)
        {
            if (n % i == 0)
            {
                if (i <= (n / i))
                {
                    FactorPair temp = new FactorPair(i, n / i);
                    tempPairs.add(temp);
                }
            }
        }
        return tempPairs;
    }

b. public int findMostPairs(Factors f)
    {
        if (this.pairs.size() > f.pairs.size())
            return this.number;
        else if (f.pairs.size() > this.pairs.size())
            return f.number;
        else
            return -1;
    }

```

FactorPair Rubric

Part (a)

+3

+1

Correctly declare an ArrayList of type FactorPair, and loop to find factors (1 through n, or 1 through $n / 2$) by examining every number for factors, without any bounds errors

+1

Finds factors correctly—looks for evenly divisible factors that are less than or equal to n / i (and does not add duplicate factors, using the = to find factors of perfect square numbers, such as 5 in the number 25)

+1

Builds FactorPair object and adds it to the ArrayList

Part (b)

+2

+1

Correctly evaluates the size of the two ArrayLists to find the object with the greater number of FactorPair objects

+1

Properly returns the number that had the most FactorPair objects or -1 if both numbers had the same number of FactorPair objects. Must use objectname.number.

4. Array2DMultiples—Canonical solution

```

public static int [][] eliminateDuplicateRows(int [][] arrWithDups)
{
    int [][] arr = arrWithDups;
    int holdRow = 0;
    int r = 1;
    while (holdRow < arr.length - 1)
    {
        while ( r < arr.length )
        {
            if (arr[holdRow][0] == arr[r][0] )
            {
                int [][] smallerArr = new int [arr.length - 1][arr[0].length];
                for (int row = 0; row < r; row ++)
                    for (int col = 0; col < arr[0].length; col++)
                        smallerArr[row][col] = arr[row][col];

                for (int row = r + 1; row < arr.length; row ++)
                    for (int col = 0; col < arr[0].length; col++)
                        smallerArr[row - 1][col] = arr[row][col];
                arr = smallerArr;
            }
            else
                r++;
        }
        holdRow ++;
        r = holdRow + 1;
    }
    return arr;
}

```

Array2DMultiples Rubric

+6

+1	Parameter arrWithDups is not modified
+1	A row is held; a loop is used to examine each subsequent row for duplicates without any bounds errors
+1	Duplicate row is identified
+1	Smaller 2D array is declared
+1	New 2D array is built correctly without duplicates
+1	New 2D array is returned

HOW TO SCORE PRACTICE TEST 2

Section I: Multiple-Choice

$$\frac{\text{Number Correct (out of 42)}}{\text{Number Correct (out of 42)}} \times 1.964 = \text{Weighted Section I Score (Do not round)}$$

Section II: Free-Response

Question 1: $\frac{\text{Number Correct (out of 7)}}{\text{Number Correct (out of 7)}} \times 2.700 = \text{Weighted Section II Score (Do not round)}$

Question 2: $\frac{\text{Number Correct (out of 7)}}{\text{Number Correct (out of 7)}} \times 2.700 = \text{Weighted Section II Score (Do not round)}$

Question 3: $\frac{\text{Number Correct (out of 5)}}{\text{Number Correct (out of 5)}} \times 2.700 = \text{Weighted Section II Score (Do not round)}$

Question 4: $\frac{\text{Number Correct (out of 6)}}{\text{Number Correct (out of 6)}} \times 2.700 = \text{Weighted Section II Score (Do not round)}$

AP Score Conversion Chart Computer Science A	
Composite Score Range	AP Score
107–150	5
90–106	4
73–89	3
56–72	2
0–55	1

Sum = $\frac{\text{Sum of Section II Scores}}{\text{Sum of Section II Scores}} \times 2.700 = \text{Weighted Section II Score (Do not round)}$

Composite Score

$$\text{Weighted Section I Score} + \text{Weighted Section II Score} = \text{Composite Score (Round to nearest whole number)}$$